

Reflexionsfragen Woche 7

Wie war die Erfahrung, die erste API von Grund auf zu bauen? Was war schwierig?

- Das eigenständige Programmieren einer REST-Schnittstelle von Beginn an war eine gewinnbringende Erfahrung, da sich dadurch das Verständnis für die dahinterstehenden HTTP-Prozesse stark verbessert hat. Schwierigkeiten bereiteten anfangs vor allem die saubere Verknüpfung der einzelnen Routing-Dateien sowie das korrekte Management asynchroner Abläufe und von ID-Pfaden in der Konsole.

Welche CRUD-Operationen waren am einfachsten zu implementieren?

- Am unkompliziertesten umzusetzen waren die Leseoperationen (GET zum Aufrufen einzelner oder aller Aufgaben) sowie der POST-Endpunkt für das Anlegen neuer Tasks, da diese Routen einem klaren Standard folgen. Hierbei mussten lediglich Daten ausgegeben oder ein neues Element an eine vorhandene Liste angehängt werden, ohne dass komplizierte Filterregeln beachtet werden mussten.

Warum ist der PATCH /transition Endpoint besonders wichtig für Kanban?

- Diese Schnittstelle ist von entscheidender Bedeutung, da sie im Vergleich zu einer einfachen, ungesteuerten Aktualisierung gezielt den Statuswechsel einer Aufgabe überwacht. Dadurch lässt sich eine automatische Prüfinstanz (Middleware) einbinden, die unzulässige Statussprünge – wie das direkte Überspringen des Teststadiums – serverseitig konsequent unterbindet.

Kanban-Board und WIP-Limits

Was ist der Sinn von WIP-Limits? Wie verhindert es Chaos im Team?

- Die Vorgabe von WIP-Limits (Work-in-Progress) dient dem Zweck, die Menge zeitgleich begonnener Tasks bewusst zu deckeln, um Überlastung durch Multitasking sowie Blockaden zu vermeiden. Chaos wird verhindert, indem das Team nach der Maxime „Aufhören anzufangen – anfangen aufzuhören“ dazu angehalten wird, angefangene Arbeiten erst gemeinschaftlich abzuschließen, bevor neue Aufgaben angegangen werden.

Wenn "In Progress" ein WIP-Limit von 3 hat - was passiert, wenn ein Team-Member versucht, eine 4. Task zu starten?

- Sowohl bei einem physischen Board als auch in einem restriktiv eingestellten digitalen System wird das Ablegen der vierten Aufgabe blockiert, um die Grenze einzuhalten. Bezogen auf unsere API führt ein solcher Versuch zu einer kontrollierten Fehlermeldung (wie einem HTTP 400 Bad Request), die dem Team

programmgesteuert oder visuell signalisiert, dass zuerst ein Engpass beseitigt werden muss.

Wie unterscheidet sich Kanban-Thinking von klassischem Wasserfall?

- Während der klassische Wasserfall-Ansatz auf unelastischen, aufeinanderfolgenden Projektphasen und einer langfristigen Planung im Vorfeld beruht, fokussiert sich Kanban auf einen kontinuierlichen, flexiblen Fluss von Ergebnissen nach dem Pull-Prinzip. Kanban ermöglicht es, jederzeit agil auf Modifikationen zu reagieren und Fortschritte schrittweise zu erzielen, anstatt das gesamte Produkt gesammelt ganz am Ende des Projekts auszuliefern.

Vergleich API und Jira Board

Wie hängen die Task-Statuses in Ihrer API mit den Jira-Spalten zusammen?

- Die in der API definierten Statusbezeichnungen (wie backlog, todo, in_progress, done) spiegeln die Spaltenanordnung auf dem Jira-Kanban-Board identisch wider. Jede Spalte auf der Benutzeroberfläche von Jira korrespondiert somit exakt mit einem zulässigen Datenzustand im Systemhintergrund der API.

Wie könnten Sie Ihre API mit Jira verbinden (API und Jira Webhook)?

- Man könnte in Jira eine automatische Benachrichtigungsfunktion (einen Webhook) einrichten, die bei jeder Positionsänderung einer Karte ein HTTP-Signal an die API übermittelt. Die API müsste hierfür eine dedizierte Empfangsschnittstelle (wie POST /api/jira-webhook) bereitstellen, um die eigenen Daten fortlaufend mit dem Zustand auf dem Jira-Board zu synchronisieren.